Lab Manual

of

Object Oriented Programming

Subject Code: 204ES Academic Year 2026-27



Name of Student : ..... Vinit Nitinbhai Dubal .....

Student ID No : ..... 25EC413 .....

Batch : ..... Batch A .....

Lab Faculty Name : ..... Mayur Sevak .....

BACHELOR OF ENGINEERING

In

Electronics and Communication

BVM Engineering College

(An Autonomous Institution)

Vallabh Vidhyanagar-388120

BVM ENGINEERING COLLEGE

(An Autonomous Institution)

Electronics and Communication Department AY: 2025-26,
Semester-III

CERTIFICATE

This is to certify that Mr. / Ms. ..... Vinit Nitinbhai Dubal ..... of Year/Sem. .. 3rd Sem .. ID No ... 25EC413 ... has satisfactorily completed his/her term work in Month of ..... for the term ending in November-2026. Number of practical certified ..... out of ..... in the subject of **204ES: Object Oriented Programming.**

Date: ..... **/** ..... **/ 2026**

Signature of Teacher

BVM ENGINEERING COLLEGE

(An Autonomous Institution)

Electronics and Communication Department AY: 2026-27, Semester-III

Object Oriented Programming (204ES)

INDEX

Name: ..... Vinit Nitinbhai Dubal ..... **ID No.:** ..... 25EC413 .....

Pr. No.	Title	Date	Sign
1.	Write a Program to display basic details of a User like Name, Roll no, Division and Address.		
2.	Write a Program to make simple calculator using Switch case.		
3.	Write a Program to Convert number of Days into Number of Months and remaining days.		
4.	Write a Program to enter marks of 6 subject and find out Total Marks and Percentage.		
5.	Write a Program to find whether the given number is even or odd.		
6.	Write a Program to find the largest among three numbers.		
7.	Write a Program to find whether given number is Armstrong or not.		
8.	Write a Program which generates table of a given number.		
9.	Write a Program which uses concept of Reference Variable.		
10.	Write a Program to swap two numbers using Call by Value method.		
11.	Write a Program to swap two numbers using Call by Reference Method.		
12.	Write a Program which uses Function as default argument.		
13.	Write a Program to calculate factorial of a number using recursion.		
14.	Write a Program to take input in an array and print its sum.		
15.	Write a Program to read string and print total number of characters.		
16.	Write a Program to create a class called ‘Student’ that has Name, ID, CPI, SPI and Semester as data Member. Also uses Proper member Function for taking and Displaying information.		
17.	Write a Program to count number of objects using static variable.		

Pr. No.	Title	Date	Sign
18.	Define a class to represent a bank account which includes the following members as data member: a) Name of Depositor b) Account Number c) Withdrawal Amount d) Balance Member Function: a) To assign initial Value b) To Deposit an amount c) To withdraw an amount after checking balance d) To display name and balance. Write the above program for handling n number of account holder using array of object.		
19.	Write a Program to demonstrate the use of Constructors and Destructors.		
20.	Write a Program which reads the complex number, copy that into other using copy constructor.		
21.	Write a Program implement function Overloading for geometric area.		
22.	Write a Program for addition of two complex number using Friend Function. (Use contractor function to initialize data member of complex class.		
23.	Write a Program to find the mean value of a given number using friend function.		
24.	Write a program to add integer value of two class objects.		
25.	Write a Program to do addition of two times objects.		
26.	Write a Program to design a class having static member function named showcount() which has the property of displaying t number of object created of the class.		
27.	Write a program which uses concept of Unary Operator (++/–) overloading.		
28.	Write a program which uses concept of Binary Operator overloading on String.		
29.	Write a program to create a Matrix and overload the '+' operator to add two matrix object.		
30.	Define two classes polar and rectangle to represent point in t polar and rectangle systems. Use conversion routines to convert from one system to the other.		
31.	Write a program to implement Single Inheritance.		
32.	Write a program to implement Multiple Inheritance.		
33.	Write a program to create File with constructor Function.		
34.	WAP to demonstrate the use of Template		

Practical 1

Aim

Write a program to display basic details of a user like Name, Roll No, Division and Address.

Algorithm

- Start.
- Declare variables for Name, Roll No, Division and Address.
- Accept input from user.
- Display all details.
- Stop.

Program

```
#include<iostream>
#include<string>
using namespace std;
int main()
{
   string name, division, address;
   int roll;
   cout<<"Enter Name: ";
   getline(cin,name);
   cout<<"Enter Roll No: ";
   cin>>roll;
   cin.ignore();
   cout<<"Enter Division: ";
   getline(cin,division);
   cout<<"Enter Address: ";
   getline(cin,address);
   cout<<"\n------ Student Details ------";
   cout<<"\nName      : "<<name;
   cout<<"\nRoll No   : "<<roll;
   cout<<"\nDivision  : "<<division;
   cout<<"\nAddress   : "<<address;
   return 0;
}
```

Sample Output

```
Enter Name: Hardik Joshi
Enter Roll No: 27
Enter Division: EC-A
Enter Address: Vadodara

------ Student Details ------
Name      : Hardik Joshi
Roll No   : 27
Division  : EC-A
Address   : Vadodara
```

Practical No. 2

Aim

To write a C++ program to perform simple arithmetic operations (Addition, Subtraction, Multiplication, Division, and Modulus) using the **switch-case** statement.

Theory

A **switch-case** statement is a multi-way decision-making statement in C++. It allows a variable or expression to be tested against multiple cases. Depending on the selected operator, the corresponding arithmetic operation is executed.

Syntax:

```
switch(expression)
{
   case value1:
       // Statements
       break;
   case value2:
       // Statements
       break;
   default:
       // Statements
}
```

Algorithm

- Start the program.
- Declare two numbers and an operator.
- Accept two numbers from the user.
- Accept an arithmetic operator (+, -, *, /, %).
- Use the switch statement to perform the selected operation.
- Display the result.
- Stop the program.

Program

```
#include <iostream>
using namespace std;
int main()
{
   int num1, num2;
   char op;
   cout << "Enter first number: ";
   cin >> num1;
   cout << "Enter second number: ";
   cin >> num2;
   cout << "Enter operator (+, -, *, /, %): ";
   cin >> op;
   switch(op)
   {
       case '+':
           cout << "Addition = " << num1 + num2;
```

```
            break;
        case '-':
            cout << "Subtraction = " << num1 - num2;
            break;
        case '*':
            cout << "Multiplication = " << num1 * num2;
            break;
        case '/':
            if(num2 != 0)
                cout << "Division = " << (float)num1 / num2;
            else
                cout << "Division by zero is not allowed.";
            break;
        case '%':
            if(num2 != 0)
                cout << "Modulus = " << num1 % num2;
            else
                cout << "Modulus by zero is not allowed.";
            break;
        default:
            cout << "Invalid Operator!";
    }
    return 0;
}
```

Sample Output 1

```
Enter first number: 48
Enter second number: 6
Enter operator (+, -, *, /, %): +
Addition = 54
```

Sample Output 2

```
Enter first number: 36
Enter second number: 8
Enter operator (+, -, *, /, %): /
Division = 4.5
```

Sample Output 3

```
Enter first number: 47
Enter second number: 6
Enter operator (+, -, *, /, %): %
Modulus = 5
```

Practical No. 3

Aim

To write a C++ program to convert a given number of days into equivalent **months** and **remaining days**.

Theory

In this program, it is assumed that **1 month = 30 days**. The total number of days entered by the user is divided by 30 to calculate the number of months, and the remainder is calculated using the modulus (%) operator to determine the remaining days.

Formula:

- Number of Months = Total Days / 30
- Remaining Days = Total Days % 30

Algorithm

- Start the program.
- Declare integer variables days, months, and remainingDays.
- Accept the total number of days from the user.
- Calculate the number of months using integer division:
  - months = days / 30
- Calculate the remaining days:
  - remainingDays = days % 30
- Display the number of months and remaining days.
- Stop the program.

Program

```cpp
#include <iostream>
using namespace std;
int main()
{
   int days, months, remainingDays;
   cout << "Enter the number of days: ";
   cin >> days;
   months = days / 30;
   remainingDays = days % 30;
   cout << "\nEquivalent Time:";
   cout << "\nMonths         = " << months;
   cout << "\nRemaining Days = " << remainingDays;
   return 0;
}
```

Sample Output 1

```
Enter the number of days: 100

Equivalent Time:
Months         = 3
Remaining Days = 10
```

Sample Output 2

```
Enter the number of days: 90

Equivalent Time:
Months         = 3
Remaining Days = 0
```

Sample Output 3

```
Enter the number of days: 200

Equivalent Time:
Months         = 6
Remaining Days = 20
```

Practical No. 4

Aim

To write a C++ program to enter the marks of six subjects and calculate the **Total Marks** and **Percentage**.

Theory

In this program, the marks of six subjects are entered by the user. The total marks are obtained by adding the marks of all six subjects, and the percentage is calculated by dividing the total marks by the number of subjects.

Formula:

- **Total Marks = Sum of marks of all six subjects**
- **Percentage = Total Marks / 6**

Assumption: Each subject is out of **100 marks**.

Algorithm

- Start the program.
- Declare six variables to store the marks of six subjects.
- Declare variables for total marks and percentage.
- Accept marks of all six subjects from the user.
- Calculate the total marks by adding all six marks.
- Calculate the percentage using the formula:
  - Percentage = Total Marks / 6.0
- Display the total marks and percentage.
- Stop the program.

Program

```cpp
#include <iostream>
using namespace std;
int main()
{
   int sub1, sub2, sub3, sub4, sub5, sub6;
   int total;
   float percentage;
   cout << "Enter marks of Subject 1: ";
   cin >> sub1;
   cout << "Enter marks of Subject 2: ";
   cin >> sub2;
   cout << "Enter marks of Subject 3: ";
   cin >> sub3;
   cout << "Enter marks of Subject 4: ";
   cin >> sub4;
   cout << "Enter marks of Subject 5: ";
   cin >> sub5;
   cout << "Enter marks of Subject 6: ";
   cin >> sub6;
   total = sub1 + sub2 + sub3 + sub4 + sub5 + sub6;
   percentage = total / 6.0;
```

```
    cout << "\nTotal Marks = " << total;
    cout << "\nPercentage = " << percentage << "%";
    return 0;
}
```

Sample Output

```
Enter marks of Subject 1: 72
Enter marks of Subject 2: 81
Enter marks of Subject 3: 65
Enter marks of Subject 4: 90
Enter marks of Subject 5: 77
Enter marks of Subject 6: 84

Total Marks = 469
Percentage = 78.1667%
```

Practical No:06

write c++ program to find the largest among three numbers.

```
#include <iostream>
using namespace std;
int main() {
    int a, b, c;
    cout << "Enter three numbers: ";
    cin >> a >> b >> c;
    if (a >= b && a >= c)
        cout << "Largest number is: " << a;
    else if (b >= a && b >= c)
        cout << "Largest number is: " << b;
    else
        cout << "Largest number is: " << c;
    return 0;
}
```

Example Input:

```
Enter three numbers: 42 17 63
```

Output:

```
Largest number is: 63
```

Practical No:07

Write a Program to find whether given number is Armstrong or not.

```
#include <iostream>
#include <cmath>
using namespace std;
int main() {
    int num, originalNum, remainder, digits = 0;
    int result = 0;
    cout << "Enter a number: ";
    cin >> num;
    originalNum = num;
    // Count the number of digits using for loop
    for (; originalNum != 0; originalNum /= 10) {
        digits++;
    }
    originalNum = num;
    // Calculate the sum of digits raised to the power of digits
    for (; originalNum != 0; originalNum /= 10) {
        remainder = originalNum % 10;
        result += pow(remainder, digits);
    }
    // Check Armstrong number
    if (result == num)
        cout << num << " is an Armstrong Number.";
    else
        cout << num << " is not an Armstrong Number.";
    return 0;
}
```

Sample Output

Input:

```
Enter a number: 9474
```

Output:

```
9474 is an Armstrong Number.
```

Practical :08

C++ Program to Generate the Table of a Given Number

```cpp
#include <iostream>
using namespace std;
int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;
    cout << "\nMultiplication Table of " << num << ":\n";
    for (int i = 1; i <= 10; i++) {
        cout << num << " x " << i << " = " << num * i << endl;
    }
    return 0;
}
```

Sample Output

Input:

```
Enter a number: 12
```

Output:

```
Multiplication Table of 12:
12 x 1 = 12
12 x 2 = 24
12 x 3 = 36
12 x 4 = 48
12 x 5 = 60
12 x 6 = 72
12 x 7 = 84
12 x 8 = 96
12 x 9 = 108
12 x 10 = 120
```

Practical :09

C++ Program Using the Concept of Reference Variable

```cpp
#include <iostream>
using namespace std;
int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;
    // Reference variable
    int &ref = num;
    cout << "\nOriginal value of num = " << num << endl;
    cout << "Value using reference variable = " << ref << endl;
    // Modify the value using reference variable
    ref = ref + 10;
    cout << "\nAfter modifying through reference variable:" << endl;
    cout << "Value of num = " << num << endl;
    cout << "Value of ref = " << ref << endl;
    return 0;
}
```

Sample Output

Input:

```
Enter a number: 40
```

Output:

```
Original value of num = 40
Value using reference variable = 40

After modifying through reference variable:
Value of num = 50
Value of ref = 50
```

Practical :10

C++ Program to Swap Two Numbers Using Call by Value

```cpp
#include <iostream>
using namespace std;
// Function using Call by Value
void swap(int a, int b) {
    int temp;
    temp = a;
    a = b;
    b = temp;
    cout << "\nInside swap() function:" << endl;
    cout << "a = " << a << ", b = " << b << endl;
}
int main() {
    int x, y;
    cout << "Enter two numbers: ";
    cin >> x >> y;
    cout << "\nBefore swapping:" << endl;
    cout << "x = " << x << ", y = " << y << endl;
    swap(x, y);
    cout << "\nAfter calling swap() function:" << endl;
    cout << "x = " << x << ", y = " << y << endl;
    return 0;
}
```

Sample Output

Input:

```
Enter two numbers: 35 60
```

Output:

```
Before swapping:
x = 35, y = 60

Inside swap() function:
a = 60, b = 35

After calling swap() function:
x = 35, y = 60
```

Practical :11

C++ program to swap two numbers using the **Call by Reference** method.

```cpp
#include <iostream>
using namespace std;
// Function to swap two numbers using call by reference
void swapNumbers(int &a, int &b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
int main()
{
    int num1, num2;
    cout << "Enter first number: ";
    cin >> num1;
    cout << "Enter second number: ";
    cin >> num2;
    cout << "\nBefore Swapping:" << endl;
    cout << "First Number = " << num1 << endl;
    cout << "Second Number = " << num2 << endl;
    swapNumbers(num1, num2);
    cout << "\nAfter Swapping:" << endl;
    cout << "First Number = " << num1 << endl;
    cout << "Second Number = " << num2 << endl;
    return 0;
}
```

Sample Output

```
Enter first number: 72
Enter second number: 18

Before Swapping:
First Number = 72
Second Number = 18

After Swapping:
First Number = 18
Second Number = 72
```

Practical :12

C++ Program: Function Using Default Arguments

```cpp
#include <iostream>
using namespace std;
// Function with default arguments
int add(int a = 10, int b = 20)
{
    return a + b;
}
int main()
{
    cout << "add() = " << add() << endl;           // Uses both defaults
    cout << "add(5) = " << add(5) << endl;         // a = 5, b = 20
    cout << "add(5, 15) = " << add(5, 15) << endl; // a = 5, b = 15
    return 0;
}
```

Sample Output

```
add() = 30
add(5) = 25
add(5, 15) = 20
```

Practical :13

C++ program to calculate the **factorial of a number using recursion**.

```cpp
#include <iostream>
using namespace std;
// Recursive function to calculate factorial
int factorial(int n)
{
    if (n == 0 || n == 1)
        return 1;
    else
        return n * factorial(n - 1);
}
int main()
{
    int num;
    cout << "Enter a number: ";
    cin >> num;
    if (num < 0)
    {
        cout << "Factorial of a negative number is not defined.";
    }
    else
    {
        cout << "Factorial of " << num << " = " << factorial(num);
    }
    return 0;
}
```

Sample Output

```
Enter a number: 6
Factorial of 6 = 720
```

Practical :14

C++ program to take input in an array and print the sum of its elements.

```cpp
#include <iostream>
using namespace std;
int main()
{
    int arr[100], n, sum = 0;
    cout << "Enter the number of elements: ";
    cin >> n;
    cout << "Enter " << n << " elements:" << endl;
    for (int i = 0; i < n; i++)
    {
        cin >> arr[i];
    }
    // Calculate sum of array elements
    for (int i = 0; i < n; i++)
    {
        sum = sum + arr[i];
    }
    cout << "Sum of array elements = " << sum << endl;
    return 0;
}
```

Sample Output

```
Enter the number of elements: 6
Enter 6 elements:
12
25
8
31
4
15
Sum of array elements = 95
```

Practical :15

simple C++ program to read a string and print the total number of characters.

```
#include <iostream>
#include <string>
using namespace std;
int main()
{
    string str;
    cout << "Enter a string: ";
    getline(cin, str);
    cout << "Total number of characters = " << str.length() << endl;
    return 0;
}
```

Sample Output

```
Enter a string: Object Oriented Programming
Total number of characters = 27
```

——————————————————————Simple Program Using Class and Object ——————————————————————

```
#include <iostream>
using namespace std;
// Class Definition
class Student
{
public:
    int rollNo;
    string name;
    // Member Function
    void getData()
    {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cout << "Enter Name: ";
        cin >> name;
    }
    void displayData()
    {
        cout << "\nStudent Details" << endl;
        cout << "Roll Number: " << rollNo << endl;
        cout << "Name: " << name << endl;
    }
};
int main()
{
    // Object Creation
    Student s1,s2;
    // Calling Member Functions
    s1.getData();
    s2.getData();
    s1.displayData();
    s2.displayData();
    return 0;
}
```

Practical :16

Here is a simple C++ program that creates a class Student with data members **Name, ID, CPI, SPI, and Semester**, along with member functions to **accept** and **display** student information.

```cpp
#include <iostream>
#include <string>
using namespace std;
class Student
{
private:
    string name;
    int id;
    float cpi, spi;
    int semester;
public:
    // Member function to accept student details
    void getData()
    {
        cout << "Enter Student Name: ";
        getline(cin, name);
        cout << "Enter Student ID: ";
        cin >> id;
        cout << "Enter CPI: ";
        cin >> cpi;
        cout << "Enter SPI: ";
        cin >> spi;
        cout << "Enter Semester: ";
        cin >> semester;
    }
    // Member function to display student details
    void displayData()
    {
        cout << "\n----- Student Information -----" << endl;
        cout << "Name      : " << name << endl;
        cout << "ID        : " << id << endl;
        cout << "CPI       : " << cpi << endl;
        cout << "SPI       : " << spi << endl;
        cout << "Semester  : " << semester << endl;
    }
};
int main()
{
    Student s;
    s.getData();
    s.displayData();
    return 0;
}
```

Sample Output

```
Enter Student Name: Neha Desai
Enter Student ID: 2043
Enter CPI: 7.92
Enter SPI: 8.35
Enter Semester: 3

----- Student Information -----
Name      : Neha Desai
ID        : 2043
CPI       : 7.92
SPI       : 8.35
Semester  : 3
```

Practical :17

Here is a simple C++ program to **count the number of objects created using a static data member**.

```cpp
#include <iostream>
using namespace std;
class Student
{
private:
    static int count;   // Static data member
public:
    // Constructor
    Student()
    {
        count++;
        cout << "Object " << count << " created." << endl;
    }
    // Static member function
    static void displayCount()
    {
        cout << "\nTotal number of objects created = " << count << endl;
    }
};
// Definition of static data member
int Student::count = 0;
int main()
{
    Student s1;
    Student s2;
    Student s3;
    Student::displayCount();
    return 0;
}
```

Sample Output

```
Object 1 created.
Object 2 created.
Object 3 created.

Total number of objects created = 3
```

Practical :18

Here is a C++ program that defines a BankAccount class and handles **n number of account holders using an array of objects**.

```
#include <iostream>
#include <string>
using namespace std;
class BankAccount
{
private:
    string name;
    int accountNo;
    float balance;
    float withdrawAmount;
public:
    // Function to assign initial values
    void assign()
    {
        cout << "\nEnter Depositor Name: ";
        cin.ignore();
        getline(cin, name);
        cout << "Enter Account Number: ";
        cin >> accountNo;
        cout << "Enter Initial Balance: ";
        cin >> balance;
    }
    // Function to deposit amount
    void deposit()
    {
        float amount;
        cout << "\nEnter amount to deposit: ";
        cin >> amount;
        balance = balance + amount;
        cout << "Amount Deposited Successfully." << endl;
    }
    // Function to withdraw amount
    void withdraw()
    {
        cout << "\nEnter amount to withdraw: ";
        cin >> withdrawAmount;
        if (withdrawAmount <= balance)
        {
            balance = balance - withdrawAmount;
            cout << "Withdrawal Successful." << endl;
        }
        else
        {
            cout << "Insufficient Balance!" << endl;
        }
    }
    // Function to display account details
    void display()
    {
        cout << "\n-----------------------------" << endl;
        cout << "Depositor Name : " << name << endl;
        cout << "Account Number : " << accountNo << endl;
        cout << "Balance        : " << balance << endl;
    }
};
int main()
```

```
{
    int n;
    cout << "Enter Number of Account Holders: ";
    cin >> n;
    BankAccount acc[n];   // Array of Objects
    // Accept details
    for (int i = 0; i < n; i++)
    {
        cout << "\nEnter Details of Account Holder " << i + 1 << endl;
        acc[i].assign();
    }
    // Deposit and Withdraw
    for (int i = 0; i < n; i++)
    {
        cout << "\nTransactions for Account Holder " << i + 1 << endl;
        acc[i].deposit();
        acc[i].withdraw();
    }
    // Display details
    cout << "\n\n----- Account Details -----" << endl;
    for (int i = 0; i < n; i++)
    {
        acc[i].display();
    }
    return 0;
}
```

Sample Output

```
Enter Number of Account Holders: 2

Enter Details of Account Holder 1

Enter Depositor Name: Karan Mehta
Enter Account Number: 2101
Enter Initial Balance: 7500

Enter Details of Account Holder 2

Enter Depositor Name: Isha Trivedi
Enter Account Number: 2102
Enter Initial Balance: 4000

Transactions for Account Holder 1

Enter amount to deposit: 2500
Amount Deposited Successfully.

Enter amount to withdraw: 4000
Withdrawal Successful.

Transactions for Account Holder 2

Enter amount to deposit: 1000
Amount Deposited Successfully.

Enter amount to withdraw: 6000
Insufficient Balance!
```

```
----- Account Details -----

----------------------------
Depositor Name : Karan Mehta
Account Number : 2101
Balance        : 6000

----------------------------
Depositor Name : Isha Trivedi
Account Number : 2102
Balance        : 5000
```

Practical :19

Here is a simple C++ program that demonstrates the use of **Constructors** and **Destructors**.

```cpp
#include <iostream>
using namespace std;
class Student
{
public:
    // Constructor
    Student()
    {
        cout << "Constructor Called: Object Created." << endl;
    }
    // Destructor
    ~Student()
    {
        cout << "Destructor Called: Object Destroyed." << endl;
    }
    void display()
    {
        cout << "Welcome to C++ Programming!" << endl;
    }
};
int main()
{
    Student s1;    // Constructor is called automatically
    s1.display();
    return 0;      // Destructor is called automatically
}
```

Sample Output

```
Constructor Called: Object Created.
Welcome to C++ Programming!
Destructor Called: Object Destroyed.
```

Practical :20

Write a Program which reads the complex number, copy that into other using copy constructor.

```
#include <iostream>
using namespace std;
class Complex
{
    float real, imag;
public:
    // Parameterized constructor
    Complex(float r = 0, float i = 0)
    {
        real = r;
        imag = i;
    }
    // Copy Constructor
    Complex(const Complex &c)
    {
        real = c.real;
        imag = c.imag;
    }
    // Function to display complex number
    void display()
    {
        cout << real;
        if (imag >= 0)
            cout << " + " << imag << "i";
        else
            cout << " - " << -imag << "i";
        cout << endl;
    }
};
int main()
{
    float real, imag;
    // Read complex number
    cout << "Enter real part: ";
    cin >> real;
    cout << "Enter imaginary part: ";
    cin >> imag;
    // Create first object
    Complex c1(real, imag);
    // Copy c1 into c2 using copy constructor
    Complex c2(c1);
    cout << "\nOriginal Complex Number: ";
    c1.display();
    cout << "Copied Complex Number: ";
    c2.display();
    return 0;
}
```

Sample Output

```
Enter real part: 3.5
Enter imaginary part: -2.5

Original Complex Number: 3.5 - 2.5i
Copied Complex Number: 3.5 - 2.5i
```

Practical :21

Write a C++ Program implementing function Overloading for geometric area.

```cpp
#include <iostream>
using namespace std;
// Area of circle
float area(float radius)
{
    return 3.14159 * radius * radius;
}
// Area of rectangle
float area(float length, float breadth)
{
    return length * breadth;
}
// Area of triangle
float area(float base, float height, int)
{
    return 0.5 * base * height;
}
int main()
{
    float r, l, b, base, h;
    cout << "Enter radius of circle: ";
    cin >> r;
    cout << "Area of Circle = " << area(r) << endl;
    cout << "\nEnter length and breadth of rectangle: ";
    cin >> l >> b;
    cout << "Area of Rectangle = " << area(l, b) << endl;
    cout << "\nEnter base and height of triangle: ";
    cin >> base >> h;
    cout << "Area of Triangle = " << area(base, h, 0) << endl;
    return 0;
}
```

Sample Output

```
Enter radius of circle: 7
Area of Circle = 153.938

Enter length and breadth of rectangle: 12 5
Area of Rectangle = 60

Enter base and height of triangle: 9 6
Area of Triangle = 27
```

Practical :22

```
#include <iostream>
using namespace std;
class Complex
{
    float real, imag;
public:
    // Constructor to initialize data members
    Complex(float r = 0, float i = 0)
    {
        real = r;
        imag = i;
    }
    // Friend function declaration
    friend Complex add(Complex, Complex);
    void display()
    {
        cout << real << " + " << imag << "i";
    }
};
// Friend function definition
Complex add(Complex c1, Complex c2)
{
    Complex c3;
    c3.real = c1.real + c2.real;
    c3.imag = c1.imag + c2.imag;
    return c3;
}
int main()
{
    float r1, i1, r2, i2;
    cout << "Enter real and imaginary part of first complex number: ";
    cin >> r1 >> i1;
    cout << "Enter real and imaginary part of second complex number: ";
    cin >> r2 >> i2;
    // Constructor initializes the objects
    Complex c1(r1, i1);
    Complex c2(r2, i2);
    // Addition using friend function
    Complex c3 = add(c1, c2);
    cout << "\nFirst Complex Number = ";
    c1.display();
    cout << "\nSecond Complex Number = ";
    c2.display();
    cout << "\nAddition = ";
    c3.display();
    return 0;
}
```

Sample Output:

```
Enter real and imaginary part of first complex number: 7 2
Enter real and imaginary part of second complex number: 3 5

First Complex Number = 7 + 2i
Second Complex Number = 3 + 5i
Addition = 10 + 7i
```

Practical :23

Write a C++ program to find the **mean value of given numbers using a friend function**.

```
#include <iostream>
using namespace std;
class Mean
{
   int n;
   float arr[100];
public:
   Mean(int count)
   {
       n = count;
   }
   void getData()
   {
       cout << "Enter " << n << " numbers: ";
       for (int i = 0; i < n; i++)
           cin >> arr[i];
   }
   friend float calculateMean(Mean);
};
float calculateMean(Mean m)
{
   float sum = 0;
   for (int i = 0; i < m.n; i++)
       sum += m.arr[i];
   return sum / m.n;
}
int main()
{
   int n;
   cout << "Enter number of values: ";
   cin >> n;
   Mean m(n);
   m.getData();
   cout << "Mean = " << calculateMean(m);
   return 0;
}
```

Sample Output

```
Enter number of values: 5
Enter 5 numbers: 8 14 21 33 44
Mean = 24
```

Practical :24

```
#include <iostream>
using namespace std;
class Time
{
    int hours, minutes, seconds;
public:
    void getTime()
    {
        cout << "Enter hours, minutes and seconds: ";
        cin >> hours >> minutes >> seconds;
    }
    void display()
    {
        cout << hours << " hours "
             << minutes << " minutes "
             << seconds << " seconds";
    }
    // Operator overloading for +
    Time operator+(Time t)
    {
        Time temp;
        temp.seconds = seconds + t.seconds;
        temp.minutes = minutes + t.minutes + temp.seconds / 60;
        temp.seconds = temp.seconds % 60;
        temp.hours = hours + t.hours + temp.minutes / 60;
        temp.minutes = temp.minutes % 60;

        return temp;
    }
};
int main()
{
    Time t1, t2, t3;
    cout << "Enter first time:\n";
    t1.getTime();
    cout << "Enter second time:\n";
    t2.getTime();
    t3 = t1 + t2;
    cout << "\nFirst Time = ";
    t1.display();
    cout << "\nSecond Time = ";
    t2.display();
    cout << "\nAddition of Two Times = ";
    t3.display();
    return 0;
}
```

Sample Output

```
Enter first time:
Enter hours, minutes and seconds: 2 50 45
Enter second time:
Enter hours, minutes and seconds: 4 35 40

First Time = 2 hours 50 minutes 45 seconds
Second Time = 4 hours 35 minutes 40 seconds
Addition of Two Times = 7 hours 26 minutes 25 seconds
```

Practical :25

```
#include <iostream>
using namespace std;
class Number
{
    int value;
public:
    // Constructor
    Number(int v = 0)
    {
        value = v;
    }
    // Operator overloading for +
    Number operator+(Number obj)
    {
        Number temp;
        temp.value = value + obj.value;
        return temp;
    }
    // Display function
    void display()
    {
        cout << "Sum = " << value << endl;
    }
};
int main()
{
    Number n1(10), n2(20), n3;
    n3 = n1 + n2;
    n3.display();
    return 0;
}
```

Output

```
Sum = 30
```

Practical :26

Write a C++ Program to design a class having static member function named showcount() which has the property of displaying the number of object created of the class.

```cpp
#include <iostream>
using namespace std;
class Demo
{
    static int count;
public:
    Demo()
    {
        count++;
    }
    static void showcount()
    {
        cout << "Number of objects created = " << count << endl;
    }
};
// Definition of static data member
int Demo::count = 0;
int main()
{
    Demo d1;
    Demo d2;
    Demo d3;
    Demo d4;
    Demo::showcount();
    return 0;
}
```

Sample Output

```
Number of objects created = 4
```

Practical :27

Write C++ a program which uses concept of Unary Operator (++/–) overloading.

```
#include <iostream>
using namespace std;
class Number
{
    int value;
public:
    // Constructor
    Number(int v)
    {
        value = v;
    }
    // Unary ++ operator overloading
    void operator++()
    {
        ++value;
    }
    // Display function
    void display()
    {
        cout << "Value = " << value << endl;
    }
};
int main()
{
    Number n(10);
    cout << "Before increment:" << endl;
    n.display();
    ++n;
    cout << "After increment:" << endl;
    n.display();
    return 0;
}
```

Sample Output

```
Before increment:
Value = 10
After increment:
Value = 11
```

Practical :28

Write a c++ program which uses the concept of Binary Operator overloading on String.

```
#include <iostream>
#include <cstring>
using namespace std;
class String
{
    char str[100];
public:
    // Constructor
    String(const char s[] = "")
    {
        strcpy(str, s);
    }
    // Binary + operator overloading
    String operator+(String s)
    {
        String temp;
        strcpy(temp.str, str);
        strcat(temp.str, s.str);
        return temp;
    }
    // Display function
    void display()
    {
        cout << str << endl;
    }
};
int main()
{
    String s1("Hello ");
    String s2("World");
    String s3;
    s3 = s1 + s2;
    cout << "First String  : ";
    s1.display();
    cout << "Second String : ";
    s2.display();
    cout << "Concatenated String : ";
    s3.display();
    return 0;
}
```

```
First String  : Hello
Second String : World
Concatenated String : Hello World
```

Practical :29

Write a c++ program to create a Matrix and overload the '+' operator to add two matrix object.

```
#include <iostream>
using namespace std;
class Matrix
{
    int a[2][2];
public:
    // Function to input matrix elements
    void getdata()
    {
        cout << "Enter matrix elements:" << endl;
        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
            {
                cin >> a[i][j];
            }
        }
    }
    // Binary + operator overloading
    Matrix operator+(Matrix m)
    {
        Matrix temp;
        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
            {
                temp.a[i][j] = a[i][j] + m.a[i][j];
            }
        }
        return temp;
    }
    // Function to display matrix
    void display()
    {
        for (int i = 0; i < 2; i++)
        {
            for (int j = 0; j < 2; j++)
            {
                cout << a[i][j] << "\t";
            }
            cout << endl;
        }
    }
};
int main()
{
    Matrix m1, m2, m3;
    cout << "Enter elements of first matrix:" << endl;
    m1.getdata();
    cout << "Enter elements of second matrix:" << endl;
    m2.getdata();
    // Add two matrix objects
    m3 = m1 + m2;
    cout << "\nFirst Matrix:" << endl;
    m1.display();
    cout << "\nSecond Matrix:" << endl;
    m2.display();
```

```
    cout << "\nResultant Matrix:" << endl;
    m3.display();
    return 0;
}
```

Sample Output

```
Enter elements of first matrix:
Enter matrix elements:
2 4
6 8
Enter elements of second matrix:
Enter matrix elements:
1 3
5 7

First Matrix:
2       4
6       8

Second Matrix:
1       3
5       7

Resultant Matrix:
3       7
11      15
```

Practical :30

```
#include <iostream>
#include <cmath>
using namespace std;
class Polar;
class Rectangle
{
    float x, y;
public:
    Rectangle(float a = 0, float b = 0)
    {
        x = a;
        y = b;
    }
    Rectangle(Polar p);        // Conversion constructor
    void display()
    {
        cout << "Rectangular Coordinates = ("
             << x << ", " << y << ")" << endl;
    }
};
class Polar
{
    float r, theta;
public:
    Polar(float radius = 0, float angle = 0)
    {
        r = radius;
        theta = angle;
    }
    Polar(Rectangle rec)       // Conversion constructor
    {
        r = sqrt(rec.x * rec.x + rec.y * rec.y);
        theta = atan2(rec.y, rec.x) * 180 / 3.14159;
    }
    friend class Rectangle;
    void display()
    {
        cout << "Polar Coordinates = ("
             << r << ", " << theta << " degrees)" << endl;
    }
};
Rectangle::Rectangle(Polar p)
{
    float angle = p.theta * 3.14159 / 180;
    x = p.r * cos(angle);
    y = p.r * sin(angle);
}
int main()
{
    Rectangle R1(3, 4);
    cout << "Original Rectangle:\n";
    R1.display();
    // Rectangle -> Polar
    Polar P1 = R1;
    cout << "\nRectangle to Polar:\n";
    P1.display();
    // Polar -> Rectangle
    Rectangle R2 = P1;
```

```
    cout << "\nPolar to Rectangle:\n";
    R2.display();
    return 0;
}
```

Sample Output

```
Original Rectangle:
Rectangular Coordinates = (3, 4)

Rectangle to Polar:
Polar Coordinates = (5, 53.1301 degrees)

Polar to Rectangle:
Rectangular Coordinates = (3, 4)
```

Practical :31

Write a C++ program to implement Single Inheritance

```
#include <iostream>
using namespace std;
// Base Class
class Student
{
protected:
    int rollNo;
    string name;
public:
    void getStudentData()
    {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cout << "Enter Name: ";
        cin >> name;
    }
    void displayStudentData()
    {
        cout << "Roll Number: " << rollNo << endl;
        cout << "Name: " << name << endl;
    }
};
// Derived Class
class Result : public Student
{
private:
    float marks;
public:
    void getResult()
    {
        cout << "Enter Marks: ";
        cin >> marks;
    }
    void displayResult()
    {
        displayStudentData();
        cout << "Marks: " << marks << endl;
    }
```

```
};
int main()
{
    Result r;
    r.getStudentData();
    r.getResult();
    cout << "\n--- Student Result ---\n";
    r.displayResult();
    return 0;
}
```

Sample Output

```
Enter Roll Number: 214
Enter Name: Mehul
Enter Marks: 91

--- Student Result ---
Roll Number: 214
Name: Mehul
Marks: 91
```

Practical :32

Write a C++ program to implement Multiple Inheritance.

```cpp
#include <iostream>
using namespace std;
// Base Class 1
class Student
{
protected:
    int rollNo;
    string name;
public:
    void getStudentData()
    {
        cout << "Enter Roll Number: ";
        cin >> rollNo;
        cout << "Enter Name: ";
        cin >> name;
    }
};
// Base Class 2
class Marks
{
protected:
    float marks;
public:
    void getMarks()
    {
        cout << "Enter Marks: ";
        cin >> marks;
    }
};
// Derived Class
class Result : public Student, public Marks
{
public:
    void display()
    {
        cout << "\n--- Student Result ---" << endl;
        cout << "Roll Number: " << rollNo << endl;
        cout << "Name: " << name << endl;
        cout << "Marks: " << marks << endl;
    }
};
int main()
{
    Result r;
    r.getStudentData();
    r.getMarks();
    r.display();
    return 0;
}
```

```
Enter Roll Number: 305
Enter Name: Dhruv
Enter Marks: 78

--- Student Result ---
Roll Number: 305
```

```
Name: Dhruv
Marks: 78
```